

Scikit-learn / Machine Learning

INDUSTRY CHEAT SHEET

A practical reference for building ML systems
that are accurate, reproducible, explainable,
and ready to move from notebook → application.

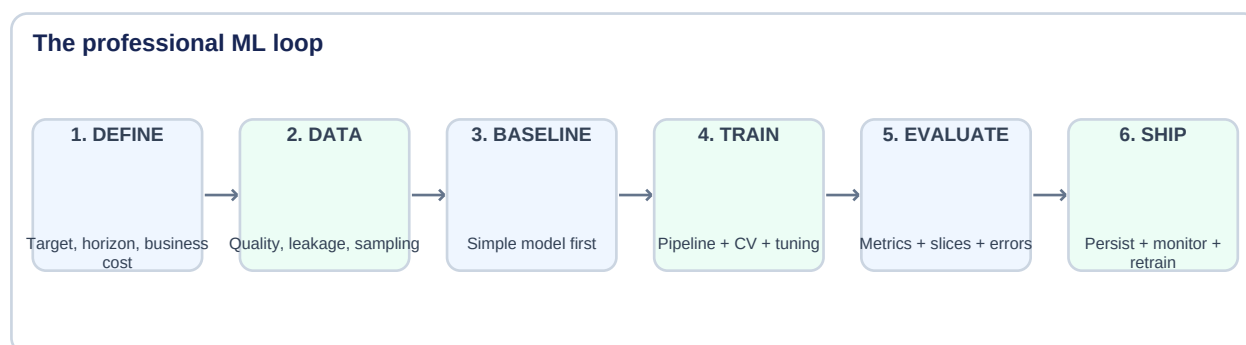


Includes: preprocessing • pipelines • algorithms • metrics • tuning • leakage •
interpretability • imbalance • persistence • production patterns • debugging

Reference baseline: [scikit-learn 1.9 documentation](#)

How to use this cheat sheet

Use it as a build-time decision guide, not as a memorization sheet. Start with the problem and data, then move through the pipeline in order. The strongest industry habit is to make every transformation, metric, experiment, and deployment assumption explicit.

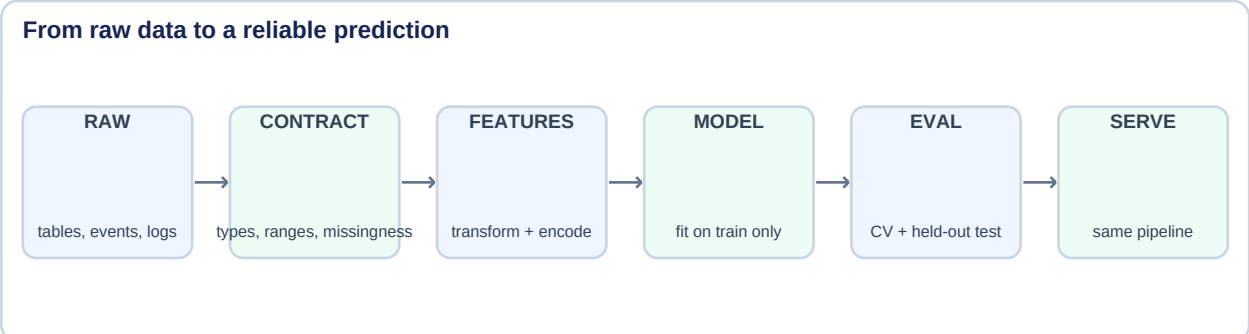


Contents

- 01** ML system mental model & workflow
- 02** Data preparation & feature engineering
- 03** Train / validation / test strategy
- 04** Preprocessing & Pipeline patterns
- 05** Algorithm selection guide
- 06** Regression reference
- 07** Classification reference
- 08** Metrics & threshold selection
- 09** Cross-validation & hyperparameter tuning
- 10** Imbalanced data, calibration & probabilities
- 11** Unsupervised learning & dimensionality reduction
- 12** Interpretability & model debugging
- 13** Production, persistence & reproducibility
- 14** Common mistakes / anti-patterns
- 15** Copy-paste project templates
- 16** Final industry checklist

01 — ML system mental model & workflow

Machine learning in an application is a chain of decisions. A model is only one component. If the target definition, split strategy, features, or evaluation metric is wrong, a stronger algorithm can make the wrong answer more confidently.



The “genius move”: write down the prediction contract before touching algorithms.

Question	What to decide
What is y?	Exactly what the model predicts. Avoid vague targets.
When is prediction made?	Define the prediction timestamp / horizon.
What data exists then?	Only features available at prediction time.
What does an error cost?	False positive vs false negative, or over/under-prediction.
What is success?	A metric tied to the real objective, not just accuracy.
What is the baseline?	Naive rule, mean/median, majority class, or current business rule.

Typical project skeleton

```
project/
├── data/                # immutable snapshots / references
├── src/
│   ├── data.py         # loading + validation
│   ├── features.py     # feature logic
│   ├── train.py        # training + CV
│   ├── evaluate.py     # metrics + error analysis
│   └── predict.py      # inference entry point
├── tests/              # unit + data + prediction tests
├── models/             # versioned artifacts
├── configs/            # experiment configuration
└── README.md           # reproducibility + usage
```

02 — Data preparation & feature engineering

Your model sees numbers, categories, or representations—not business meaning. Feature engineering is the translation layer between the real world and the estimator.

Fast data-quality checklist

Check	Questions	Typical action
Schema	Are columns typed correctly?	cast dtypes; enforce schema
Missingness	Random, structural, or informative?	impute; add indicator when justified
Duplicates	Are rows repeated?	deduplicate or define why duplicates exist
Outliers	Real events or data errors?	cap/transform/remove only with reason
Target	Is y noisy or leaked?	audit label generation
Time	Does order matter?	time-aware split; lag features
Groups	Are multiple rows from same entity?	group-aware split/CV

Numerical features

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, RobustScaler

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
    ("scaler", StandardScaler()), # useful for linear/SVM/KNN
])

# RobustScaler can help with extreme values.
# Tree ensembles usually do not require scaling.
```

Categorical features

```
from sklearn.preprocessing import OneHotEncoder

cat_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])
```

High-cardinality categories: don't blindly one-hot everything.

Options include grouping rare categories, target/frequency encoding (with leakage-safe fitting), hashing, or using a model/representation designed for high-cardinality data. The right choice depends on sample size, cardinality, latency, and how the feature is generated in production.

03 — Train / validation / test strategy

The split is part of the model. Choose it to imitate how the application will see unseen data.

Situation	Preferred split / CV
IID tabular rows	train_test_split + KFold / StratifiedKFold
Classification	StratifiedKFold when class balance matters
Same user/customer appears many times	GroupKFold / StratifiedGroupKFold
Time series / future prediction	TimeSeriesSplit or custom forward-chaining split
Very small dataset	Repeated CV / nested CV when appropriate
Rare positive class	Stratified CV + suitable metrics

Golden rule

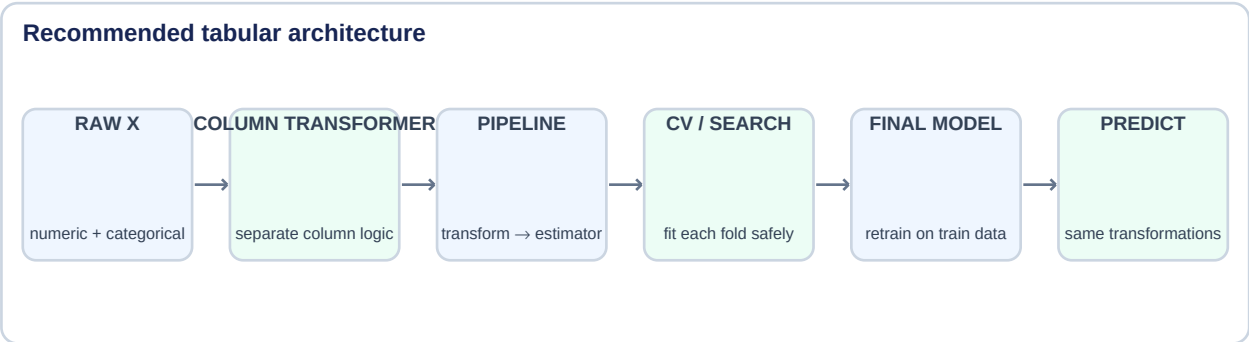
Split first. Learn preprocessing, feature selection, imputation statistics, dimensionality reduction, and model parameters from training folds only. Put learned transformations inside a Pipeline/ColumnTransformer so CV can fit them correctly per fold.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
# Keep X_test locked until the end.
# Use CV on X_train for model selection.
```

When leakage hides inside time

Suppose you predict whether an account will churn next month. A feature such as “number of support tickets in the next 30 days” may be highly predictive—but it is unavailable at prediction time. This is temporal leakage even if the code looks perfectly valid.

04 — Preprocessing & Pipeline patterns



The production-friendly pattern

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

numeric = ["age", "income", "tenure"]
categorical = ["plan", "region", "channel"]

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]), numeric),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical),
])

model = Pipeline([
    ("prep", preprocessor),
    ("clf", LogisticRegression(max_iter=2000))
])

model.fit(X_train, y_train)
pred = model.predict(X_test)
```

Why this matters

Benefit	Practical consequence
Consistency	Training and inference use identical transformations.
Leakage resistance	CV fits learned transforms only on each training fold.
Deployment	Persist one object instead of hand-recreating preprocessing.
Tuning	Search parameters across preprocessing + model together.
Testing	You can test the full prediction contract end-to-end.

05 — Algorithm selection guide

Model-selection map

LINEAR
fast, interpretable,
strong baseline

TREES
nonlinear tabular,
little preprocessing

BOOSTING
often excellent tabular
performance

KNN / SVM
distance/margin; scale
matters

NEURAL
usually outside sklearn
core sweet spot

Start simple. Escalate only when the error pattern justifies it.

Model family	Use when	Watch-outs
LinearRegression / Ridge / Lasso / ElasticNet	continuous target; baseline; coefficients matter	linearity; scaling for regularization; outliers
LogisticRegression	classification baseline; probabilities; interpretability	scale numeric features; nonlinear patterns need features
DecisionTree	rules, interactions, explainability	overfits easily; tune depth/leaf size
RandomForest / ExtraTrees	robust tabular baseline; nonlinear interactions	larger models; probabilities may need calibration
HistGradientBoosting	strong tabular performance; nonlinear effects	tuning; inspect missing/category handling and constraints
SVC	medium-sized data; margin-based classification	scaling; can be expensive as n grows
KNeighbors	small/medium data; local similarity	scaling; inference cost grows with data
KMeans	segmentation / clustering with roughly spherical clusters	choose k; scaling; sensitive to geometry
PCA	compression, denoising, visualization	scale first when feature units differ; components are combinations

A strong default for tabular applications

Compare a simple linear baseline against a tree ensemble / gradient boosting model. Use the same train/test contract and the same CV folds. If the complex model wins only on training but not validation, you have an overfitting problem—not a model-selection victory.

06 — Regression reference

Task	Good starting point	Useful next step
Numeric target, mostly linear	Ridge / ElasticNet	feature engineering or boosting
Nonlinear tabular	RandomForest / HistGradientBoosting	tune depth/learning rate/leaves
Heavy-tailed target	transform y when justified	robust metrics / quantile methods
Different cost for under vs over	custom metric / transformed target	quantile regression / business-specific loss

Metrics

Metric	Interpretation	Use carefully when
MAE	average absolute error	large errors are not disproportionately costly
MSE	squared error; punishes large misses	outliers can dominate
RMSE	error in target units; emphasizes large errors	you want large misses to hurt
R ²	variance explained relative to mean baseline	do not confuse with business usefulness
MAPE	percentage error	target can be near zero → unstable

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```
pred = model.predict(X_test)
mae = mean_absolute_error(y_test, pred)
rmse = np.sqrt(mean_squared_error(y_test, pred))
r2 = r2_score(y_test, pred)
```

```
print({"MAE": mae, "RMSE": rmse, "R2": r2})
```

Regression genius check

Always compare against a naive baseline (mean/median or a business rule), inspect residuals, and evaluate important slices. A model with a better RMSE can still be worse for the business if it systematically fails on a high-value segment.

07 — Classification reference

Confusion matrix — the four numbers behind classification metrics

Actual +

Actual −

Predicted +

Predicted −

TP

FN

FP

TN

Metric	Formula intuition	When it shines
Accuracy	$(TP+TN)/all$	balanced classes + equal error cost
Precision	$TP/(TP+FP)$	false positives are expensive
Recall	$TP/(TP+FN)$	false negatives are expensive
F1	harmonic mean of precision/recall	balance P and R
ROC AUC	ranking quality across thresholds	general discrimination view
PR AUC	precision-recall tradeoff	rare positive class
Log loss	quality of predicted probabilities	probability quality matters

Never choose a classification threshold by habit.

The default 0.5 threshold is a decision rule, not a law of nature. Choose a threshold from business costs, capacity constraints, expected value, or a desired recall/precision level. Tune the threshold on validation data—not on the locked test set.

```
proba = model.predict_proba(X_val)[: , 1]
threshold = 0.35
pred = (proba >= threshold).astype(int)
# Evaluate precision/recall/cost at this threshold.
```

08 — Metrics & threshold selection

Think of metrics as measurement instruments. The right metric answers the business question; the wrong metric can make a bad model look excellent.

Goal	Metric family	Practical note
Catch as many positives as possible	Recall / FN cost	optimize threshold and monitor missed positives
Only act when highly confident	Precision	watch capacity and missed positives
Rare event ranking	PR AUC	usually more informative than accuracy
Probability quality	Log loss / Brier / calibration	check reliability, not only ranking
Regression with costly big errors	RMSE / MSE	outliers matter by design
Regression with robust average error	MAE	easier to explain in target units

Confusion-matrix derived metrics

```
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
f1 = 2 * precision * recall / (precision + recall)
```

Scoring in cross-validation

```
from sklearn.model_selection import cross_validate, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_validate(
    model, X_train, y_train, cv=cv,
    scoring={
        "roc_auc": "roc_auc",
        "pr_auc": "average_precision",
        "f1": "f1",
        "precision": "precision",
        "recall": "recall",
    }, n_jobs=-1
)
```

09 — Cross-validation & hyperparameter tuning

Hyperparameter search is not a substitute for a good split, good features, or a correct metric. Tune only after the baseline pipeline is valid.

Search	Use when	Rule of thumb
GridSearchCV	small, meaningful parameter grid	great for a few high-impact parameters
RandomizedSearchCV	many parameters / broad ranges	often better search efficiency
Successive halving	many candidates, limited budget	eliminate weak configs early

```
from sklearn.model_selection import RandomizedSearchCV
```

```
search = RandomizedSearchCV(
    estimator=model,
    param_distributions={
        "clf__C": [0.01, 0.1, 1, 10, 100],
        "clf__class_weight": [None, "balanced"],
    },
    n_iter=10, scoring="average_precision", cv=5,
    n_jobs=-1, random_state=42, refit=True,
)
search.fit(X_train, y_train)
best_model = search.best_estimator_
```

Nested CV when you need an unbiased estimate after extensive tuning

Outer CV estimates generalization; inner CV performs tuning. It is computationally heavier, but useful when dataset size is limited and the model-selection process itself could otherwise bias the estimate.

Track every experiment

Record	Examples
Data	dataset snapshot / query version / row count
Features	feature version / exclusions / transformations
Split	random seed / groups / time window / folds
Model	class + hyperparameters
Metric	primary metric + secondary metrics
Result	mean CV score + variability + test score
Artifact	model version + environment versions

10 — Imbalanced data, calibration & probabilities

Class imbalance is not automatically a problem. The real issue is whether the minority class is important and whether your metric/decision rule reflects that importance.

Technique	Idea	Caution
Class weights	penalize minority errors more	may change probability interpretation
Threshold tuning	change action rule without retraining	must tune on validation data
Resampling	alter class distribution for training	do inside CV pipeline; never leak test data
PR AUC	focus on positive-class retrieval	depends on prevalence
Calibration	make probabilities match observed frequencies	ranking and calibration are different goals

```
from sklearn.calibration import CalibratedClassifierCV
```

```
calibrated = CalibratedClassifierCV(
    estimator=base_model, method="sigmoid", cv=5
)
calibrated.fit(X_train, y_train)
```

Probability sanity check

If the model says “0.8 probability” for 100 similar cases, roughly 80 should be positive in a well-calibrated population. A model can have excellent ranking (AUC) but poor probability calibration.

11 — Unsupervised learning & dimensionality reduction

Goal	Method	Key decisions
Customer / item segmentation	KMeans	number of clusters; scaling; stability
Find density-based groups	DBSCAN / related methods	distance scale; density parameters
Compress correlated features	PCA	scaling; number of components
Visualize high-dimensional data	PCA / manifold methods	visualization ≠ proof of clusters
Find anomalies	IsolationForest / density methods	define what anomaly means operationally

```

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

```

```

pca_model = make_pipeline(
    StandardScaler(),
    PCA(n_components=0.95)
)
X_reduced = pca_model.fit_transform(X_train)

```

Clustering evaluation

Internal scores such as silhouette can help compare clusterings, but they do not guarantee business meaning. Validate clusters against stability, domain interpretation, downstream usefulness, and whether cluster assignments remain sensible on new data.

12 — Interpretability & model debugging

Interpretability is not decoration. It helps detect leakage, unstable features, spurious relationships, and segment-specific failures.

Tool	What it tells you	Best use
Coefficients	direction/magnitude under model assumptions	linear models; sanity checks
Permutation importance	performance drop when a feature is shuffled	model-agnostic importance
Partial dependence	average model response as feature changes	global effect inspection
ICE	individual response curves	heterogeneous effects
Error slicing	performance by subgroup / range / time	production risk discovery

```
from sklearn.inspection import permutation_importance
```

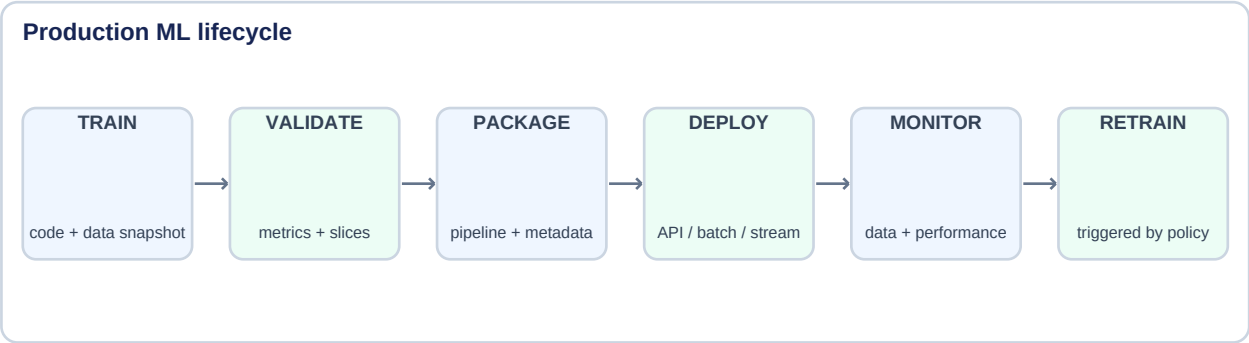
```
r = permutation_importance(
    model, X_val, y_val,
    n_repeats=20, random_state=42, scoring="roc_auc"
)
importance = r.importances_mean
```

Do not interpret importance as causality.

A feature can be predictive because it is a proxy, because another variable causes both, or because of data collection behavior. Importance answers “does this feature help this model on this evaluation data?”—not “does changing this feature cause the outcome to change?”

13 — Production, persistence & reproducibility

A production model needs a reproducible artifact, environment, input contract, monitoring plan, and rollback strategy.



Persistence choices

Format	When to consider it	Important warning
ONNX	serve without Python; supported estimator	not every sklearn model converts
skops.io	security-conscious sklearn persistence	inspect trusted types
jorlib	trusted Python environment; large model objects	pickle-based; never load untrusted files
pickle	trusted, controlled Python environments	arbitrary code execution risk on load

Save metadata with the model

```
metadata = {
    "model_version": "2026-08-31_001",
    "sklearn_version": "...",
    "python_version": "...",
    "feature_schema_version": "...",
    "training_data_snapshot": "...",
    "primary_metric": "...",
    "cv_score": "...",
    "random_seed": 42,
}
```

Deployment rule

Persist the full preprocessing + estimator pipeline whenever possible. Do not recreate preprocessing separately in a web API by copying notebook code.

14 — Common mistakes / anti-patterns

Anti-pattern	Why it fails	Fix
Scale before train/test split	test statistics leak into training	Pipeline + fit on train only
Feature selection before CV	folds see information from each other	put selection inside Pipeline
Tune on the test set	test becomes part of model selection	lock test; use validation/CV
Accuracy on rare events	majority-class model can look great	PR AUC / recall / precision / cost
Only report one score	hides variance and subgroup failures	CV distribution + slices + errors
Use future information	temporal leakage	feature availability audit
Ignore groups	same entity leaks across train/test	GroupKFold / grouped split
Manual preprocessing in production	train/serve skew	persist Pipeline
Load unknown pickle/joblib	arbitrary code execution risk	trusted artifacts / skops / ONNX where suitable
Chase the fanciest model	complexity without evidence	baseline → diagnose → escalate

The 10-second leakage test

For every feature, ask: “Could I know this exact value at the instant the prediction is requested?” If the answer is no, remove it or redesign it.

15 — Copy-paste project templates

Template A — classification

```
import numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

prep = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), numeric_cols),
    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_cols),
])

pipe = Pipeline([
    ("prep", prep),
    ("model", LogisticRegression(max_iter=2000))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_validate(
    pipe, X_train, y_train, cv=cv,
    scoring=["average_precision", "roc_auc", "f1"], n_jobs=-1
)
pipe.fit(X_train, y_train)
```

Template B — regression

```
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.model_selection import KFold, cross_validate

pipe = Pipeline([
    ("prep", preprocessor),
    ("model", HistGradientBoostingRegressor(random_state=42))
])

cv = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_validate(
    pipe, X_train, y_train, cv=cv,
    scoring=["neg_mean_absolute_error", "neg_root_mean_squared_error"],
    n_jobs=-1
)
```

Template C — prediction API boundary

```
# Keep the API contract simple:
# input JSON -> validated dataframe -> persisted pipeline -> output JSON

def predict(payload: dict) -> dict:
    x = validate_payload(payload)
    p = model.predict(X)[0]
    return {"prediction": float(p), "model_version": MODEL_VERSION}
```

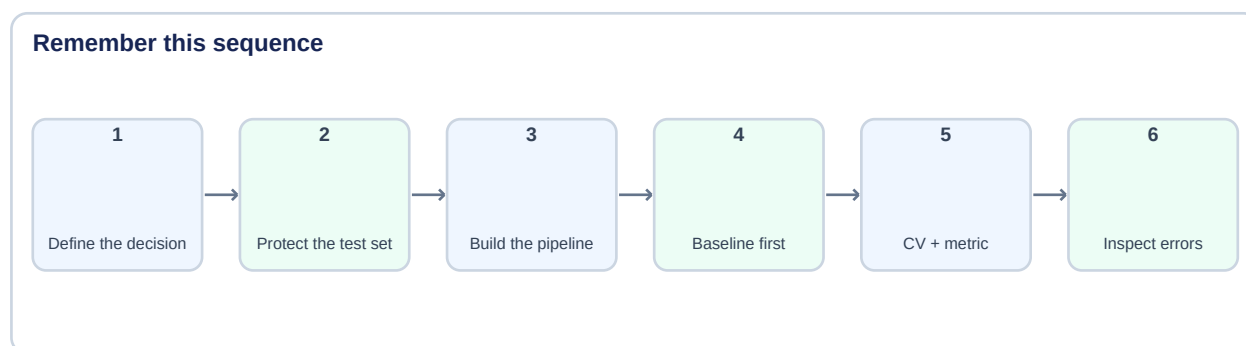
16 — Final industry checklist

Stage	Done when...
Problem	target, prediction time, action, and business cost are explicit
Data	schema, missingness, duplicates, outliers, labels, and provenance are checked
Split	train/validation/test mimics real deployment and respects groups/time
Features	every feature is available at prediction time
Baseline	a simple baseline is measured and stored
Pipeline	preprocessing + estimator are one reproducible object
CV	model selection uses appropriate folds and a primary metric
Tuning	only high-impact parameters are searched
Evaluation	test is untouched until final evaluation
Errors	confusion matrix/residuals and important slices are inspected
Interpretability	feature effects / importance are sanity-checked
Deployment	artifact + versions + schema are packaged
Monitoring	data drift, prediction distribution, latency, and business KPI are tracked
Retraining	trigger and rollback criteria are defined

The professional mindset

A “genius” ML workflow is not about using the most algorithms. It is about making fewer silent mistakes: correct target, correct split, leakage-safe pipeline, appropriate metric, controlled experiments, interpretable errors, reproducible artifact, and monitoring after deployment.

One-page mental model



Useful sklearn imports to remember

```

from sklearn.model_selection import train_test_split, KFold, StratifiedKFold, GroupKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import Ridge, LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import cross_validate, RandomizedSearchCV
from sklearn.inspection import permutation_importance, PartialDependenceDisplay
  
```

Reference & further reading

This cheat sheet is designed as a practical industry reference and is aligned with the current scikit-learn documentation consulted for this edition.

Topic	Official reference
User Guide	scikit-learn.org/stable/user_guide.html
Common pitfalls & leakage	scikit-learn.org/stable/common_pitfalls.html
Model persistence	scikit-learn.org/stable/model_persistence.html
Inspection / interpretability	scikit-learn.org/stable/inspection.html
Permutation importance	scikit-learn.org/stable/modules/permutation_importance.html

Version note

The official documentation used for this edition identifies scikit-learn 1.9.0 as the current stable documentation baseline. APIs can evolve; before shipping an application, pin and test the exact versions used in your environment.

Core principle

If you remember only one thing: make the training process reproduce the prediction process. The same feature logic, same preprocessing, same schema assumptions, and a versioned model artifact should flow from training to production.